ASSIGNMENT-3(pl/sql)

1.Program to find the greatest of 3 numbers.

Objective :

To find greatest of three numbers first declare 3 variables a,b and c and using if statement & “and” keyword in between the variables perform comparison operation to find greatest among the numbers

General Syntax:

```
if(a>b and a>c)then;
dbms_output.put_line(“a is greatest”);
```

1.Program to find the greatest of 3 numbers.

```
declare
 a number;
 b number;
 c number;
 begin
 a:=&a;
 b:=&b;
  c:=&c;
 if(a>b) and (a>c) then
 dbms_output.put_line('a is greatest');
  elsif(b>a) and (b>c) then
dbms_output.put_line('b is greatest');
 else
dbms_output.put_line('c is greatest');
 end if;
 end;
 /
```

OUTPUT:

```
Enter value for a: 5
old   2: a number:=&a;
new   2: a number:=5;
Enter value for b: 2
old   3: b number:=&b;
new   3: b number:=2;
sum is7

PL/SQL procedure successfully completed.
```

2.Write program to generate multiplication table of a given number

Program Title:
Write program to generate multiplication table of a given number

Objective :
To perform multiplication first declare a variable a and use another variable i to perform iterative operation using the for loop

General Syntax:
for I in i..n loop
dbms_output.put_line(“multiplication is”a*i);

```
  declare
    a number;
    i number;
    begin
    a:=&a;
    for i in 1..10 loop
    dbms_output.put_line('value is'||a*i);
    end loop;
   end;
 /
```

OUTPUT:

```
Enter value for n: 3
old   2:  n number:=&n;
new   2:  n number:=3;
3*1=3
3*2=6
3*3=9
3*4=12
3*5=15
3*6=18
3*7=21
3*8=24
3*9=27
3*10=30

PL/SQL procedure successfully completed.
```

3.Write a pl/sql code to insert values in a tables with columns no,square and square root.

Objective :

First create a table using create command as create table SQ(no number,s number,sr number); and then in the pl/sql program declare the column names with their datatypes and using the formulas find out the square root and square of the number

```
  no:=&no;
   s:=no*no;
   sr:=sqrt(no)
```

The values into the table are inserted using the insert command

```
 insert into sq values(no,s,sr);
```

General Syntax:

```
create table SQ(no number,s number,sr number);
insert into sq values(no,s,sr);

declare
   p number:=&p;
   q number;
   r number;
   begin
   dbms_output.put_line('number is'||p);
   q:=p*p;
   dbms_output.put_line('square of a number is'||q);
   r:=sqrt(p);
  dbms_output.put_line('squareroot of a number is'||r);
  insert into sq values(p,q,r);
 end;
/
```

OUTPUT:

```
Enter value for p: 4
old   2: p number:=&p;
new   2: p number:=4;
number is4
square of a number is16
squareroot of a number is2

PL/SQL procedure successfully completed.

SQL> select * from sq;

        NO     SQUARE SQUAREROOT
---------- ---------- ----------
         4         16          2
```

4.Program to retrieve information from emp table where sal>2000.

Objective:
To retrieve information from a table we can use cursor.Cursor is a pointer to memory location which contains information necessary for processing.We can fetch the required columns sal,ename and empno into the cursor for the program here and obtain the values required.

Example: Explicit Cursors

General Syntax:
cursor c is select ename,empno,sal from emp where sal>2000;

```
 declare
   name emp.ename%type;
   no emp.empno%type;
   s emp.sal%type;
   i number;
   cursor c is select ename,empno,sal from emp where sal>2000;
   begin
   open c;
   select count(*) into i from emp where sal>2000;
  for j in 1..i loop
  fetch c into name,s,no;
 dbms_output.put_line('salary of '||name||' is '||s);
 end loop;
 close c;
 end;
  /
```

OUTPUT:

```
JONES  2975
BLAKE  2850
CLARK  2450
SCOTT  3000
KING  5000
FORD  3000

PL/SQL procedure successfully completed.
```

5.Program to display count of employees with respect to deptno and display the count and deptno.

Objective:

To display the count of employees with respect to their deptno we use cursor.Cursor is a pointer to a memory location which contains information necessary for processing.We declare a count cnt and deptno dno variable here and into the cursor we fetch the count and deptno variables declared.

General Syntax:
cursor c is select count(*),deptno from emp group by deptno;

```
SQL> declare
  cnt number;
  3  i number;
  4  D emp.deptno%type;
  5  cursor c is select count(*),deptno from emp group by deptno;
  6  begin
  7  open c;
  8  select count(distinct(deptno)) into i from emp;
  9  for j in 1..i loop
 10  fetch c into cnt,D;
 11  dbms_output.put_line('count '||cnt||' deptno '||D);
 12  end loop;
 13  close c;
 14  end;
 15  /
```

OUTPUT:

```
6  30
5  20
3  10
```

6. Create a cursor to get top five salaries from emp table.

Objective:
To obtain the top five salaries from emp table we use a cursor which acts as a pointer to memory location for processing information.we declare the variables required here as declare
name emp.ename%type;
s emp.sal%type;
using cursor we fetch the values into name and s variables

General Syntax:
cursor c is select ename,sal from emp order by sal desc;

```
 1  declare
 2  name emp.ename%type;
 3  s emp.sal%type;
 4  i number;
 5  cursor c is select ename,sal from emp order by sal desc;
 6  begin
 7  open c;
 8  for i in 1..5 loop
 9  fetch c into name,s;
10  dbms_output.put_line('salary of  '||name||'  is  '||s);
11  end loop;
12  close c;
13* end;
14  /
```

OUTPUT:

```
KING5000
FORD3000
SCOTT3000
JONES2975
BLAKE2850

PL/SQL procedure successfully completed.
```

7. Program to implement userdefined and built in exceptions.

Objective:
To obtain the difference between userdefined and built in exception we take an example of finding whether a particular department belongs to the table department or not here.Userdefined exception should be declared in the declaration section and defined later whereas built in need not be declared but should be defined.
General Syntax:declare

```
   Declaration section
 begin
   Exception section
 exception
 WHEN ex_name1 THEN
    Error handling statements
end;
eg:declare
not_exists exception;


 1  declare
 2  d number;
 3  not_exists exception;
 4  cursor c is select deptno from dept where deptno=d;
 5  begin
```

```
 6  d:=&d;
 7  open c;
 8  fetch c into d;
 9  if c%found then
10  dbms_output.put_line('Department exists');
11  else
12  raise not_exists;
13  end if;
14  close c;
15  exception
16  when not_exists then
17  dbms_output.put_line('Department not exists');
18* end;
```

OUTPUT:

```
Enter value for d: 10
old   6: d:=&d;
new   6: d:=10;
dept exists

PL/SQL procedure successfully completed
```

8.Program to get difference in between implicit and explicit cursors.

Objective:

To differentiate between implicit and explicit cursor consider the department table and column deptno as example.

Implicit cursors are created by default when DML statements like, INSERT, UPDATE, and DELETE statements are executed. They are also created when a SELECT statement that returns just one row is executed. Explicit cursors must be created when you are executing a SELECT statement that returns more than one row.

General syntax:

```
if SQL%found then  //implicit cursor
dbms_output.put_line(‘ ‘);
if c%found then     //explicit cursor
dbms_output.put_line(‘ ‘);
```

```
declare
```

```
  d number;
  begin
  d:=&d;
  select deptno into d from dept where deptno=d;
  if SQL%found then
  dbms_output.put_line('Department exists');
  else
  raise no_data_found;
 end if;
 exception
 when no_data_found then
 dbms_output.put_line('Department not exists');
 end;
15/
```

OUTPUT:

```
Enter value for d: 2
old   4:  d:=&d;
new   4:  d:=2;
dept doesnot exists

PL/SQL procedure successfully completed.
```

9. Trigger to convert the case of ename on before insert of emp.

Trigger to convert the case of ename on before insert of emp

Objective:

We have to create a trigger that converts the case of ename i.e lowercase to uppercase before inserting into emp table.A trigger is a procedure that is automatically invoked by DBMS in response to specified change in database

General Syntax:

```
create or replace trigger case before insert
  on emp1 for each row
:new.ename:=upper(:new.ename);
```

```
  1  create or replace trigger case before insert
  2  on emp1 for each row
  3  begin
  4  :new.ename:=upper(:new.ename);
  5* end;
Trigger created.
```

OUTPUT:

```
SQL> INSERT INTO E1 VALUES(10,'abc','mam',10,'10-nov-89',45,56,10);
1 row created.
SQL> select * from e1;
    EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM     DEPTNO
---------- ---------- --------- ---------- --------- ---------- ---------- ----------
      7369 SMITH      CLERK           7902 17-DEC-80        800                  20
      7499 ALLEN      SALESMAN        7698 20-FEB-81       1600        300         30
      7521 WARD       SALESMAN        7698 22-FEB-81       1250        500         30
      7566 JONES      MANAGER         7839 02-APR-81       2975                  20
      7654 MARTIN     SALESMAN        7698 28-SEP-81       1250       1400         30
      7698 BLAKE      MANAGER         7839 01-MAY-81       2850                  30
      7782 CLARK      MANAGER         7839 09-JUN-81       2450                  10
•             7788 SCOTT      ANALYST         7566 09-DEC-82       3000                  20
      7839 KING       PRESIDENT            17-NOV-81       5000                  10
      7844 TURNER     SALESMAN        7698 08-SEP-81       1500          0         30
      7876 ADAMS      CLERK           7788 12-JAN-83       1100                  20
    EMPNO ENAME      JOB              MGR HIREDATE         SAL       COMM     DEPTNO
---------- ---------- --------- ---------- --------- ---------- ---------- ----------
```

```
7900 JAMES      CLERK          7698 03-DEC-81       950                30
7902 FORD       ANALYST        7566 03-DEC-81       3000               20
7934 MILLER     CLERK          7782 23-JAN-82       1300               10
  10 abc        mam            10 10-NOV-89         45        56       10
  10 ABC        mam            10 10-NOV-89         45        56       10
16 rows selected.
```

10 Consider a table where the values of sysdate and timestamp should get automatically inserted into orderdate and ordertimes respectively.

Objective:

We have to create a trigger using in which if we insert values into order table then they should automatically be inserted in orderdate and ordertime table. A trigger is a procedure that is automatically invoked by DBMS in response to specified change in database

Order (Customer_name varchar2(9), orderno integer, Itemno integer,item_name varchar2(9), , Quantity integer(9))

Order_date date_of_order

Order-time time_of _order

General syntax:

```
create or replace trigger trig_date after insert or delete on order1   for each row
  begin
 if inserting then
 insert into order_date values(sysdate);
 insert into order_time values(SYSDATE);
 elsif deleting then
 delete from order_date;
 delete from order_time;
 end if;
 end;
/
```

Trigger created.

OUTPUT:

```
QL> SELECT * FROM ORDERS;

                ID DESCRIPTION
-------------------- --------------------------------------------------
                90 PRINTER

SQL> SELECT * FROM ORDER_DATE;

DATES
---------
23-JAN-26

SQL> SELECT * FROM ORDER_TIME;

TIME
---------------------------------------------------------------------------
23-JAN-26 11.38.34.000000 AM

SQL> INSERT INTO ORDERS VALUES(900,'PRINTERS');

1 row created.

SQL> SELECT * FROM ORDER_TIME;

TIME
---------------------------------------------------------------------------
23-JAN-26 11.38.34.000000 AM
23-JAN-26 11.40.32.000000 AM
```

```
SQL> SELECT * FROM ORDER_DATE;

DATES
---------
23-JAN-26
23-JAN-26

SQL> DELETE FROM ORDERS WHERE ID=90;

1 row deleted.

SQL> SELECT * FROM ORDER_DATE;

no rows selected

SQL> SELECT * FROM ORDER_TIME;

no rows selected
```

SQL> CREATE TABLE STUDENT_GRADES(GRADE NUMBER(9));

Table created.

RAISE AN APPLICATION ERROR FOR VIOLATING TRIGGER CONSTRAINT on table Student_Grades(Stud_id integer, Stud_name varchar2(9), Subject_name, Grade number(9));

```
SQL> CREATE TRIGGER validate_grade
  2  BEFORE INSERT ON student_grades
  3  FOR EACH ROW
```

```
  4  BEGIN
  5     IF :new.grade < 0 OR :new.grade > 100 THEN
  6        RAISE_APPLICATION_ERROR(-20001, 'Invalid grade value.');
  7     END IF;
  8  END;
  9  /
```

Trigger created.

```
SQL> INSERT INTO STUDENT_GRADES VALUES(90);
```

1 row created.

```
SQL> INSERT INTO STUDENT_GRADES VALUES(900);
INSERT INTO STUDENT_GRADES VALUES(900)
            *
ERROR at line 1:
ORA-20001: Invalid grade value.
ORA-06512: at "FAHMINA.VALIDATE_GRADE", line 3
ORA-04088: error during execution of trigger
'FAHMINA.VALIDATE_GRADE'
```

11 A program to find factorial of a given number.

Objective:

To find factorial of a given number we use functions.A function is a module that performs one or more actions but will return a value

General syntax:

```
create or replace function fact(n in number)
return number is
```

```
 1  create or replace function fact(n in number)
  2  return number is
  3  f number:=1;
  4  begin
  5  for i in 1..n loop
  6  f:=f*i;
  7  end loop;
  8  return f;
  9* end fact;
Function created.
SQL> select fact(4) from dual;
```

OUTPUT:

```
FACT(4)
----------
       24
```

12.A program to check whether a number is palindrome or not.

Objective:

To check whether a given number is palindrome or not we use procedure.A procedure is a module that performs one or more actions.It doesnot return any value.

General Syntax:

```
create or replace procedure pal2(s1 in varchar2) is
   s2 varchar2(20);
   s3 varchar2(20);
```

```
1  create or replace procedure pal2(s1 in varchar2) is
  2  s2 varchar2(20);
  3  s3 varchar2(20);
  4  begin
  5  for i in reverse 1..length(s1) loop
  6  s2:=substr(s1,i,1);
  7  s3:=s3||s2;
  8  end loop;
```

```
 9  if s1=s3 then
10  dbms_output.put_line('Palindrome');
11  else
12  dbms_output.put_line('Not palindrome');
13  end if;
14* end pal2;
```

OUTPUT:

Procedure created.

SQL> execute palindrome(121);

number is palindrome

PL/SQL procedure successfully completed.

SQL> execute palindrome(123);

not palindrome

PL/SQL procedure successfully completed.

13 A procedure to accept to deptno as parameter and display total salary, no.of employees,maximum salary and average salary of the department.

Objective:

We create a procedure pay for example which takes dno as parameter and using aggregate functions we find the required queries..A procedure is a module that performs one or more actions.It doesnot return any value.

General Syntax:

create or replace procedure pay(dno in number) is

```
1  create or replace procedure sample1 (deptno in number) is
2  a number;
3  b number;
4  c number;
5  d number;
6  begin
7  select sum(sal), count(*), max(sal),avg(sal) into a,b,c,d from emp where deptno=10;
8  dbms_output.put_line('total salary='||a||'no of employees='||b||'max salary='||c||'average
salary='||d);
9  end sample1;
Procedure created.
```

OUTPUT:

```
SQL> execute sample1(10);
total salary=8750no of employees=3max salary=5000average
salary=2916.6666666666666666666666666666666666667

PL/SQL procedure successfully completed.
```

14.Write a program to implement package with overloading of function and procedure.

Objective:

We make use of package to show overloading of procedure sum for example.Package can have procedures and functions in it.

General Syntax:

```
create or replace package overload is

  procedure sum(a number, b number);

   procedure sum(a number, b number, c number);
```

```
create or replace package overload is

  2   procedure sum(a number, b number);

  3   procedure sum(a number, b number, c number);

  4*  end;

SQL> /

Package created.

  1  create or replace package body overload as
  2  procedure sum(a number, b number, c number) is
  3  d number;
  4  begin
  5  d:=a+b+c;
  6  dbms_output.put_line('Sum of 3 nos'||d);
  7  end sum;
  8  procedure sum( a number, b number) is
  9  c number;
 10  begin
 11  c:=a+b;
 12  dbms_output.put_line('Sum of 2 numbers is'||c);
 13  end sum;
14* end;
```

OUTPUT:

```
SQL> select overload.sum(2,3) from dual;
```

```
OVERLOAD.SUM(2,3)
-----------
         5

SQL>ed

  1* select overload.sum (2,3,5) from dual
SQL> /

OVERLOAD.SUM(2,3,5)
-------------
          10
```

ASSIGNMENT-3

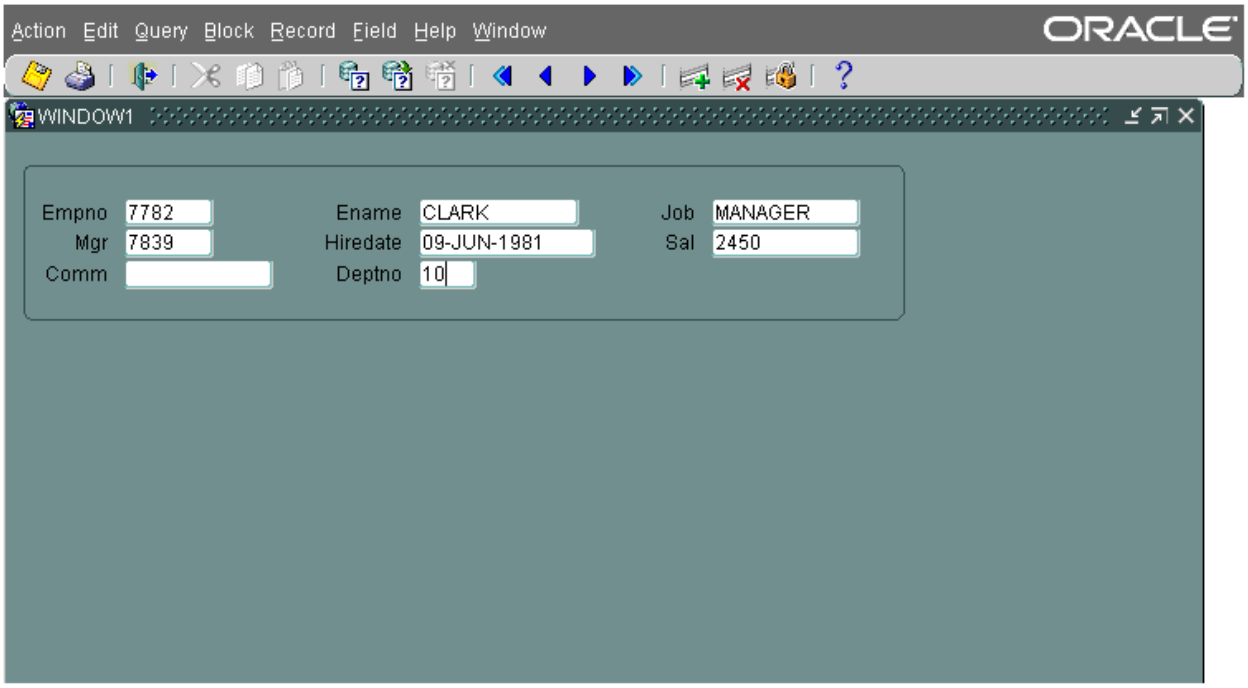
1. To create a simple form using oracle form builder.

Steps:

1. Open oracle form builder.
2. Start OC4J instance.
3. From the menu bar on the appeared window select tools and choose Data block wizard option.
4. Browse the required table of which form has to be created by clicking on the browse button.
5. A connect dialog box appears, get connected to the data base server by providing a valid login Id and password.
6. Select any table from the list of tables of which you want to build a form.
7. Run the form by selecting run option available on the toolbar.

8. A simple form is created with the specified column names.

Output:



2. To create a simple form with push button using oracle form builder.

Steps:

1. Open oracle form builder.
2. Start OC4J instance.
3. From the menu bar on the appeared window select tools and choose Data block wizard option.
4. Browse the required table of which form has to be created by clicking on the browse button.
5. A connect dialog box appears, get connected to the data base server by providing a valid login Id and password.
6. Select any table from the list of tables of which you want to build a form.
7. Add required number of push buttons to the form by selecting option available on the left side of the form.
8. Select a push button and right click the mouse,from the available list of options select Propery Palette.

9. A new window appears which posses all the properties of selected push button. Rename the push button label as “Save”.

10. Again select the same push button and right click the mouse.

- Now from the available list of options select PL/SQL editor.
- A trigger windoe appears choose **when-button-pressed** option from the available list of options.
- Click ok.
- A new window appears where we need to type the code for action to be performed when the push button is pressed.

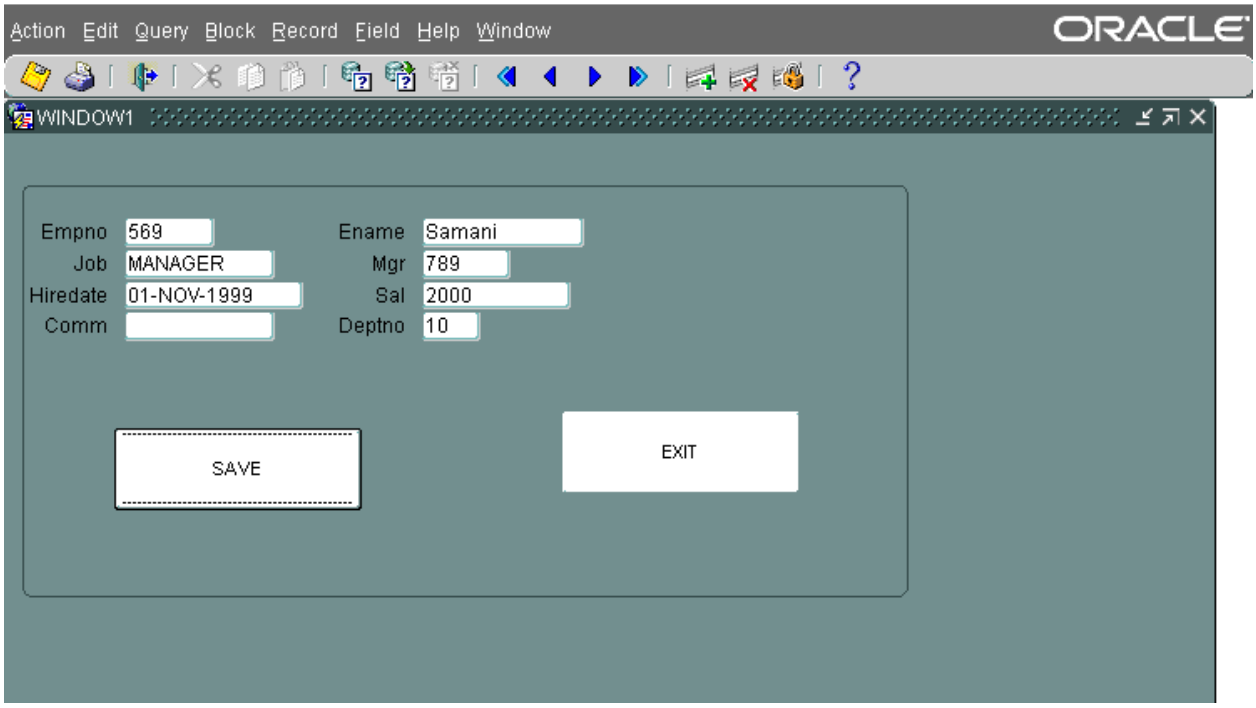
Eg: ”COMMIT” for save**.**

11. Repeat the steps **9.** and **10.** To create another push button i.e “Exit”. It has the code “**EXIT_FORM**”.

12. Run the form by selecting run option available on the toolbar.

13. A simple form is created with the specified column names.

Output:



3. To create a simple report using oracle report builder.

Steps:

1. Open oracle report builder.
2. Start OC4J instance.
3. Select use a report wizard from the available options on the window.
4. Select query builder button that appears in the window that appears immediately after choosing the data source.
5. A connect dialog box appears, get connected to the data base server by providing a valid login Id and password.
6. Select any table say emp table from the available list of data tables.Provide necessary functions to the columns attribute,

Eg. Count,average etc

7. Go on clicking the next button until the report is generated.

Output:

Your Title - Windows Internet Explorer

C:\Documents and Settings\Admin\Local Settings\Temp\docroot\MODULE100803300.htm

File Edit View Favorites Tools Help

Favorites | Internet Explorer cannot dis... | Suggested Sites | Free Hotmail | Get more Add-ons

Your Title

Page Safety Tools

YOUR COMPANY Inc.

Navigation goes here
Navigation Item

Empno	Ename	Job	Mgr	Hiredate	Sal	Comm	Deptno
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	09-DEC-82	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	12-JAN-83	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
569	Samani	MANAGER	789	01-NOV-99	2000		10
23		halwai			20063		
					Average: 3149.294117647058823529		
					Minimum: 800		
					Maximum: 20063		
Count: 17							

Done | My Computer | 100%